

Pilares do CI/CD Moderno

Práticas essenciais para desenvolvimento de software ágil,
confiável e seguro.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Sumário

- Introdução e Pilares do CI/CD
- Testes Automatizados no Pipeline
- Infraestrutura e Pipelines como Código
- Monitoramento, Observabilidade e Shift Left
- Verificação de Conhecimento e Reflexão

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Pilares do CI/CD Moderno

A integração contínua (CI) e a entrega contínua (CD) são fundamentais para o desenvolvimento de software moderno. Este paradigma não se resume a ferramentas, mas a um conjunto de práticas que garantem velocidade, confiabilidade e segurança. Os pilares principais incluem a **Automação de Testes** (validação rigorosa a cada commit), **Infraestrutura como Código (IaC)** (reprodutibilidade e consistência), **Pipelines como Código** (versionamento do ciclo de vida), **Observabilidade** (monitoramento proativo pós-deploy) e o conceito de *Shift Left* (segurança e qualidade desde o início do ciclo de desenvolvimento).



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Testes Automatizados no Pipeline

Testes Unitários

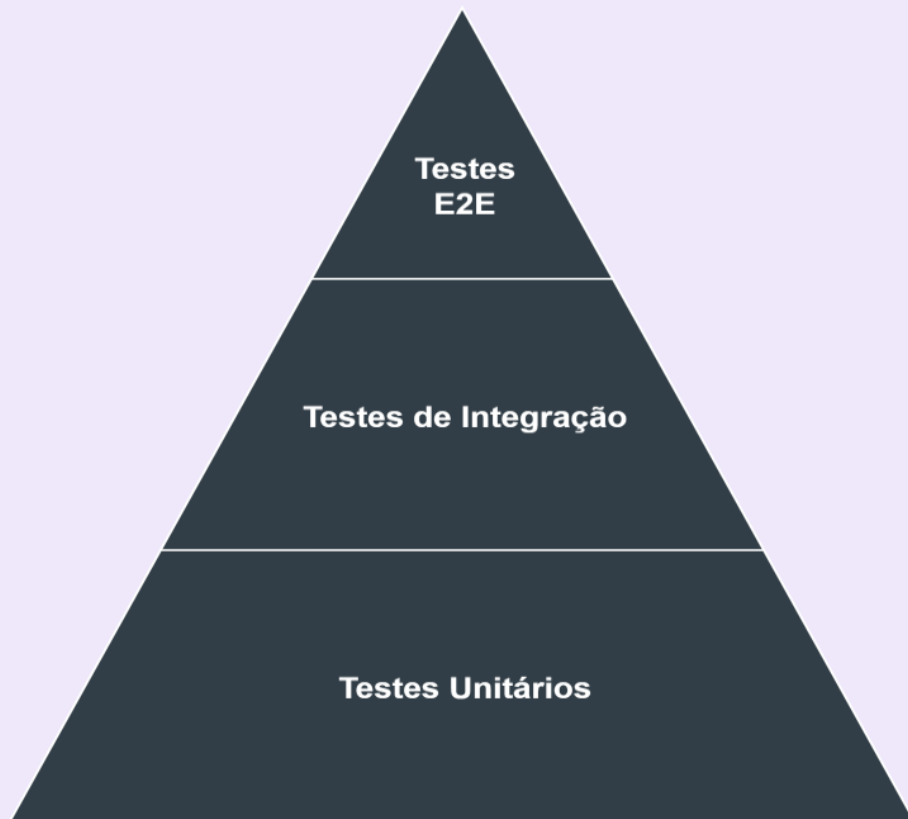
Verificam a menor parte testável do código de forma isolada. São a base da pirâmide, executados com alta frequência para garantir a **qualidade do código** rapidamente.

Testes de Integração

Validam a comunicação e a interação entre diferentes módulos ou serviços. Focam em garantir que as partes do sistema funcionem juntas, revelando problemas de *interface*.

Testes E2E (End-to-End)

Simulam a experiência real do usuário no sistema completo, desde a interface até o banco de dados. Garantem que o fluxo de trabalho crítico funciona como esperado em um ambiente próximo ao de produção.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Cenário Prático: Testes Automatizados

Este exemplo demonstra a aplicação de testes automatizados em um cenário de integração contínua, usando ferramentas como Jest, Mocha e GitHub Actions.

Contexto	Detalhes do Exemplo
Cenário	Desenvolvedor altera o sistema de login em Node.js e faz o <i>*push*</i> para o GitHub.
Execução	GitHub Actions detecta o <i>*push*</i> , sobe um <u>contêiner Docker</u> temporário e <u>instala dependências</u> .
Processo	Execução de <code>`npm run test`</code> (50 testes unitários e de integração).
Resultado	Se houver erro na validação de e-mail, o <i>*pipeline*</i> trava, notifica no Slack e bloqueia o <i>*Merge Request*</i> .

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Infraestrutura como Código (IaC)

A Infraestrutura como Código (IaC) gerencia servidores, redes e bancos de dados através de arquivos de configuração versionados. Isso garante que os ambientes de desenvolvimento e produção sejam idênticos, eliminando o problema do 'funciona na minha máquina'. O IaC permite provisionamento automatizado, rastreamento de mudanças e idempotência, assegurando um estado final consistente.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



IaC na Prática: Terraform

Arquivo .tf

O arquivo `.tf` descreve declarativamente instâncias **EC2**, regras de **VPC** e **RDS** no Terraform, definindo o estado desejado da infraestrutura.

Comando ``apply``

O *engine* do Terraform lê o estado desejado do arquivo `.tf` e provisiona a infraestrutura na nuvem em questão de minutos, criando ou modificando recursos conforme necessário.

Consistência Total

A utilização de IaC com Terraform garante que o ambiente de homologação seja um espelho exato da produção, eliminando erros manuais e inconsistências. Ferramentas como **Ansible**, **AWS CloudFormation** e **Pulumi** também são usadas para IaC.



Ícone de instâncias EC2 da AWS, base do aTerraform para infraestrutura.

INSTITUIÇÃO EXECUTORA



Terraform ``apply`` provisions cloud infrastructure from code.

COORDENADORA



APOIO



Pipelines como Código

- **Arquivos de Configuração:** A definição do processo de CI/CD reside em arquivos como ``.gitlab-ci.yml``, ``.Jenkinsfile`` ou ``.GitHub Actions`` (YAML), versionados junto ao código-fonte.
- **Rastreabilidade:** Todas as mudanças no fluxo de entrega, como qualquer outro código, são submetidas a *Code Review* e registradas no histórico do repositório.
- **Padronização:** Facilita a replicação e manutenção de fluxos de trabalho consistentes em múltiplos projetos e equipes dentro de uma organização.
- **Autonomia:** Permite que os desenvolvedores ajustem e otimizem o processo de entrega sem dependerem exclusivamente da equipe de operações.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Exemplo Prático: Pipeline as Code

1

Modificar Arquivo

O engenheiro edita o arquivo `.gitlab-ci.yml` na raiz do projeto para alterar o fluxo, tratando o pipeline como um *artefato de software versionável*.

2

Incluir Nova Etapa

Inclui uma chamada para ferramenta de análise estática (SAST) como o SonarQube no pipeline, adicionando uma camada de segurança.

3

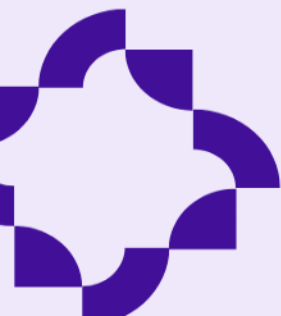
Revisar Código

A alteração passa por **Peer Review** por outros desenvolvedores, garantindo a qualidade e consistência antes de entrar em vigor.

4

Executar Automatizadamente

O pipeline agora verifica falhas de segurança em cada novo *commit* de forma transparente, reforçando a segurança contínua da aplicação.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



MINISTÉRIO DA
CIÊNCIA, TECNOLOGIA
E INOVAÇÃO



Monitoramento e Observabilidade

Métricas

Dados numéricos agregados ao longo do tempo (ex: taxa de erro, latência, uso de CPU). São cruciais para entender tendências e a performance geral do sistema, fornecendo uma visão macro.

Logs

Registros textuais detalhados de eventos discretos que ocorreram no sistema. Fornecem informações granulares sobre o que aconteceu em um ponto específico no tempo, facilitando a depuração de problemas específicos.

Traces

Rastreamento do caminho de uma requisição através de múltiplos microserviços em um ambiente distribuído. Permitem a identificação de gargalos de desempenho e a 'causa raiz' de falhas complexas.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Ferramentas para Resposta a Incidentes

Prometheus: Coleta de Métricas

O *Prometheus* é um sistema de monitoramento e alerta que coleta métricas de alvos configurados em intervalos definidos, avalia regras de expressão e dispara alertas se certas condições forem atendidas. É fundamental para a **agregação de dados** em tempo real.

Grafana: Visualização de Alertas

O *Grafana* oferece painéis visuais customizáveis para monitorar métricas. Ele exibe alertas visuais em tempo real, por exemplo, quando o uso de CPU ultrapassa 90%, facilitando a identificação rápida de anomalias no sistema.

Datadog (APM): Rastreamento de Aplicações

O *Datadog* (Application Performance Monitoring - APM) foca na performance das aplicações. Ele identifica o rastreamento de requisições e aponta funcionalidades lentas, como uma query ineficiente no banco de dados, permitindo otimizações precisas.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Ferramentas para Resposta a Incidentes

ELK Stack: Logs e Pesquisa

A *ELK Stack* (Elasticsearch, Logstash, Kibana) é um conjunto de ferramentas para gerenciamento de logs. O Logstash coleta, processa e encaminha logs, o Elasticsearch armazena e indexa, e o Kibana visualiza e pesquisa, essencial para **análise forense** de incidentes.

PagerDuty: Notificação Proativa

O *PagerDuty* é uma plataforma de gerenciamento de incidentes que atua na **notificação e escalonamento**. Ele dispara alertas automáticos para a equipe de plantão via celular ou outros canais antes mesmo que o cliente perceba a falha, garantindo uma resposta rápida.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



A Cultura Shift Left



Colaboração: Desenvolvimento e Operações trabalhando juntos em um ambiente moderno.

- **Colaboração:** Desenvolvimento e Operações trabalham juntos desde o planejamento, integrando perspectivas de *build* e *run*.
- **Feedback Precoce:** Detectar erros no editor de código é infinitamente mais barato que corrigi-los em produção, impactando diretamente o *Time To Resolution* (TTR).
- **Responsabilidade Compartilhada:** O time é dono do código do início ao fim (conceito *You build it, you run it*), promovendo o senso de propriedade.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Prevenção em Tempo Real

Os desenvolvedores usam extensões no VS Code (ex: Snyk, SonarLint) que alertam sobre vulnerabilidades, como *SQL Injection*, enquanto o código é escrito. Isso garante a identificação precoce de *bugs* e *vulnerabilities*.

Integração no Planejamento

Especialistas em segurança e infraestrutura participam ativamente das reuniões de *Planning* e *Sprints*, garantindo que a arquitetura do sistema já nasça **segura** e **performática**. Ferramentas como Snyk, SonarLint e Checkmarx são cruciais nesse processo.



INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Considerando o contexto de desenvolvimento e operações, qual das seguintes afirmações descreve corretamente a prática de ****Infraestrutura como Código (IaC)****?

- A É o processo de escrever scripts de teste unitário para validar a integridade de um banco de dados.
- B Refere-se exclusivamente ao uso de plataformas de contêineres como Docker para isolar aplicações no ambiente local do desenvolvedor.
- C É uma técnica de monitoramento que rastreia logs de erro em tempo real em ambientes de produção de alta demanda.
- D Consiste em gerenciar e provisionar recursos de rede, servidores e serviços de infraestrutura por meio de arquivos de configuração versionados.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Considerando o contexto de desenvolvimento e operações, qual das seguintes afirmações descreve corretamente a prática de ****Infraestrutura como Código (IaC)****?

- ✗ É o processo de escrever scripts de teste unitário para validar a integridade de um banco de dados.
- ✗ Refere-se exclusivamente ao uso de plataformas de contêineres como Docker para isolar aplicações no ambiente local do desenvolvedor.
- ✗ É uma técnica de monitoramento que rastreia logs de erro em tempo real em ambientes de produção de alta demanda.
- ✓ Consiste em gerenciar e provisionar recursos de rede, servidores e serviços de infraestrutura por meio de arquivos de configuração versionados.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Considerando os pilares apresentados, qual você acredita ser o maior desafio para implementar CI/CD em uma empresa tradicional: as ferramentas técnicas (como Terraform/Jenkins) ou a mudança na cultura das equipes (Shift Left)? Por quê?



Pondere sobre os aspectos técnicos e humanos envolvidos na transformação.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO



Conclusão

- CI/CD integra práticas como testes unitários (Jest, Mocha) e IaC (Terraform) para automação e consistência desde o início do ciclo.
- Pipelines como Código (GitHub Actions, GitLab CI) e Observabilidade (Grafana, Datadog) rastreiam o ciclo de vida e monitoram sistemas.
- A cultura Shift Left antecipa segurança e qualidade com ferramentas (Snyk, SonarLint), reduzindo custos de correção em produção.
- CI/CD é um paradigma de práticas DevOps que garante entrega contínua, agilidade, confiabilidade e segurança de software.

INSTITUIÇÃO EXECUTORA



COORDENADORA



APOIO

